

Coding Test — Senior Backend Developer

Untuk Kandidat Estimasi waktu: **5–8 jam** (boleh dibagi 2 sesi). Kamu boleh menggunakan AI coding tools. Yang kami nilai: kemampuan kamu mendesain sistem yang tahan banting, memahami trade-off, dan menghasilkan kode yang production-ready — bukan sekadar berjalan.

Konteks

Platform disbursement kami tumbuh pesat. Volume transaksi meningkat dan kami mulai menghadapi masalah nyata: **race conditions pada approval**, **duplikasi request dari client yang unstable**, dan **audit trail yang tidak lengkap**. Kamu diminta membangun ulang API disbursement dengan arsitektur yang siap menghadapi masalah ini.

Bagian 1 — Desain & Keputusan Arsitektur

Tulis jawaban di `ARCHITECTURE.md` di root project. Dinilai terpisah dari kode.

Jawab dua pertanyaan berikut (masing-masing 150–250 kata):

- 1.1 Idempotency** Bagaimana kamu memastikan `POST /disbursements` tidak membuat duplikasi jika client mengirim request yang sama dua kali (misalnya karena timeout lalu retry)? Jelaskan mekanisme yang kamu pilih dan di mana kamu menyimpan state idempotency-nya.
- 1.2 Concurrency & Locking** Dua admin menekan tombol "Approve" pada disbursement yang sama secara bersamaan. Bagaimana sistem kamu mencegah race condition? Jelaskan pendekatan yang kamu pilih (optimistic locking, pessimistic locking, atau lainnya) dan trade-off-nya.

Bagian 2 — Implementasi API

2.1 Autentikasi & Otorisasi

- `POST /auth/login` — menerima `username` dan `password`, mengembalikan **access token** (15 menit) dan **refresh token** (7 hari)
- `POST /auth/refresh` — menukar refresh token dengan access token baru
- `POST /auth/logout` — invalidasi refresh token

Semua endpoint selain `/auth/*` wajib dilindungi JWT middleware.

Seed 3 user:

Username	Password	Role
<code>superadmin</code>	<code>superadmin123</code>	<code>superadmin</code>

Username	Password	Role
admin	admin123	admin
operator	operator123	operator

Ketentuan role:

- `operator` — hanya bisa membuat dan melihat disbursement
- `admin` — semua akses operator + bisa mengubah status disbursement
- `superadmin` — semua akses admin + bisa menghapus dan melihat audit log

2.2 Idempotency

Endpoint `POST /disbursements` wajib mendukung header `Idempotency-Key`:

```
Idempotency-Key: <uuid-v4>
```

- Jika key yang sama digunakan dalam 24 jam, kembalikan response identik tanpa memproses ulang
- Jika key tidak disertakan, endpoint tetap berfungsi normal (tanpa jaminan idempotency)
- Response harus menyertakan header `X-Idempotent-Replayed: true` jika response berasal dari cache

2.3 Disbursement — Core Endpoints

`GET /disbursements`

Parameter	Type	Keterangan
page	number	Default: 1
limit	number	Default: 20, max: 100
search	string	Partial match pada <code>recipient_name</code>
status	string	PENDING, APPROVED, REJECTED
date_from	date	Format YYYY-MM-DD
date_to	date	Format YYYY-MM-DD
sort_by	string	created_at atau amount (default: created_at)
sort_order	string	asc atau desc (default: desc)

Contoh response:

```
{
  "success": true,
  "data": [ ... ],
  "meta": {
```

```
{
  "page": 1,
  "limit": 20,
  "total": 284,
  "total_pages": 15
}
```

GET `/disbursements/:id`

Mengembalikan semua field disbursement, termasuk `updated_at` dan `approved_by` (nullable).

POST `/disbursements`

Request body:

```
{
  "recipient_name": "Budi Santoso",
  "account_number": "1234567890",
  "bank_code": "BCA",
  "amount": 1250000,
  "note": "Pembayaran supplier"
}
```

Ketentuan:

- `recipient_name`, `account_number`, `bank_code`, `amount` — wajib
- `amount` harus bilangan bulat positif, minimal `10000`
- Status awal selalu `PENDING`
- `admin_fee` dihitung otomatis: `2500` jika `amount < 5.000.000`, `5000` jika `amount >= 5.000.000`
- `created_by` diambil dari JWT token
- Mendukung `Idempotency-Key` header (lihat 2.2)

PATCH `/disbursements/:id/status`

```
{
  "status": "APPROVED",
  "note": "Sudah diverifikasi"
}
```

Ketentuan:

- Hanya `admin` dan `superadmin` — tolak dengan `403` jika bukan
- `status` hanya boleh `APPROVED` atau `REJECTED`
- Disbursement yang sudah `APPROVED` atau `REJECTED` tidak bisa diubah — kembalikan error dengan pesan yang jelas

- Endpoint ini harus **concurrency-safe** — jika dua request simultan masuk untuk disbursement yang sama, hanya satu yang berhasil, yang lain mendapat error yang jelas
- `approved_by` diisi dari JWT token

`DELETE /disbursements/:id`

- Hanya `superadmin`
- Hanya bisa menghapus disbursement berstatus `PENDING`
- **Soft delete** — gunakan kolom `deleted_at`, bukan hapus dari database

2.4 Audit Log

`GET /audit-logs` — hanya untuk `superadmin`

Setiap perubahan pada disbursement (create, update status, delete) harus dicatat di **tabel terpisah**:

```
{
  "id": "LOG-001",
  "entity_id": "DSB-001",
  "action": "status_changed",
  "actor": "admin",
  "before": { "status": "PENDING" },
  "after": { "status": "APPROVED" },
  "created_at": "2025-06-12T08:00:00Z"
}
```

Filter yang didukung: `entity_id`, `action`, `date_from`, `date_to`. Pagination standar.

Audit log **tidak boleh memblokir operasi utama** — jika penulisan audit log gagal, operasi disbursement tetap berhasil (log kegagalan ini di server log).

2.5 Structured Logging

Setiap request harus menghasilkan log terstruktur (JSON) yang minimal berisi:

```
{
  "level": "info",
  "timestamp": "2025-06-12T08:00:00Z",
  "request_id": "550e8400-e29b-41d4-a716-446655440000",
  "method": "PATCH",
  "path": "/disbursements/DSB-001/status",
  "status_code": 200,
  "latency_ms": 42,
  "user": "admin"
}
```

`request_id` digenerate per request dan harus di-propagate ke semua log dalam satu request (handler, service, repository). Response harus menyertakan header `X-Request-ID` dengan nilai yang sama.

Fitur Bonus *(tidak wajib)*

- ☐ `POST /disbursements/batch` — buat beberapa disbursement sekaligus (partial success OK)
 - ☐ Rate limiting per user: `POST /disbursements` maks 30 req/menit, endpoint lain maks 120 req/menit
 - ☐ `GET /health` — status database + HTTP `503` jika database tidak dapat dijangkau
 - ☐ Docker + `docker-compose.yml` untuk menjalankan aplikasi + database
 - ☐ Swagger / OpenAPI documentation
-

Ketentuan Teknis

Language & Framework

Pilih salah satu:

- **Go** — Gin, Chi, atau Fiber
- **Node.js** — TypeScript + Express.js atau Fastify

Database

PostgreSQL atau MySQL (**wajib** untuk versi senior — SQLite tidak cukup untuk concurrency test).

ORM / query builder:

- Go → GORM, sqlx, atau `database/sql`
- Node.js → Prisma, Drizzle, atau Knex.js

Migration file wajib — schema harus mencerminkan keputusan desain kamu (indexing, constraint).

Yang Wajib

- **JWT** access + refresh token
- **Idempotency** untuk `POST /disbursements`
- **Concurrency-safe** status update
- **Soft delete** dengan kolom `deleted_at`
- **Audit log** di tabel terpisah
- **Structured logging** dengan request ID propagation
- **Layered architecture** — handler, service, repository, model dipisah
- **Environment variable** untuk semua konfigurasi
- **Migration file**
- **Unit test** — minimal untuk:
 - Logika kalkulasi `admin_fee`
 - Validasi perubahan status (termasuk idempotency handler)

Yang Tidak Perlu

- Integrasi payment provider nyata
- Email / notifikasi
- Deploy ke server production

Kriteria Penilaian

Desain Sistem (25%)

- ☐ `ARCHITECTURE.md` menjawab kedua pertanyaan dengan reasoning yang solid
- ☐ Keputusan di kode konsisten dengan `ARCHITECTURE.md`
- ☐ Database schema (index, constraint, soft delete) mencerminkan pemahaman performa dan integritas data

Ketahanan & Correctness (35%)

- ☐ Idempotency bekerja benar — request kedua dengan key sama mengembalikan response identik tanpa side effect
- ☐ Race condition pada approval di-handle — concurrent request menghasilkan hanya satu yang berhasil
- ☐ Refresh token bisa menukar access token baru; logout menginvalidasi refresh token
- ☐ Audit log dicatat di tabel terpisah, tidak memblokir operasi utama
- ☐ Soft delete berjalan — data tidak hilang, hanya ditandai `deleted_at`

Kualitas Kode (25%)

- ☐ Tidak ada logika bisnis di layer handler/controller
- ☐ Error dari database dan dependency di-handle secara eksplisit — tidak ada panic/crash yang tidak tertangani
- ☐ Request ID di-propagate ke semua log dalam satu request
- ☐ Unit test bermakna — ada edge case, bukan hanya happy path

API Design (15%)

- ☐ HTTP status code tepat di semua kondisi
- ☐ Format response konsisten di semua endpoint
- ☐ README lengkap: cara setup, daftar endpoint, schema database, catatan keputusan teknis

Cara Submit

1. Upload ke **GitHub / GitLab** (repository publik atau invite akun kami)
2. Kirim link ke **[email rekrutmen]** subject: `[BE-SENIOR] Nama Kamu`
3. Sertakan:
 - `ARCHITECTURE.md` — jawaban Bagian 1
 - `README.md` — cara menjalankan, daftar endpoint, schema database, catatan teknis
 - Migration file(s)

Deadline: 4 hari kalender sejak test dikirimkan

FAQ

Q: Bagaimana cara mensimulasikan concurrent request untuk race condition? A: Tidak perlu — kami akan mengujinya saat review. Tugas kamu adalah memastikan implementasinya benar secara kode.

Q: Refresh token harus disimpan di mana? A: Terserah — database atau in-memory. Jelaskan pilihan dan trade-off-nya di `ARCHITECTURE.md`.

Q: Apakah unit test harus comprehensive? A: Tidak perlu 100% coverage. Fokus pada logika bisnis kritis: idempotency handler, concurrency lock, dan kalkulasi fee. Test yang hanya memverifikasi fungsi dipanggil tidak bernilai.

Q: Boleh menggunakan library tambahan? A: Boleh, selama bisa menjelaskan alasan dan trade-off-nya di README.

Q: Apakah harus ada migration file? A: Ya, wajib. Minimal satu migration file yang membuat semua tabel. Schema SQL di README sebagai alternatif juga diterima.